

Divide and Recombine GLM on the BRFSS Dataset

Authors: Kevin Ji, Blake Xu, Sangey Rinchen

Our chosen paper

- *“Divide and recombine approaches for fitting logistic regression to large-scale health surveillance data: application to diabetes risk prediction in BRFSS”*
- Analyzing large datasets(~1M) with statistical models can have high computation cost
- Most relevant for health surveillance and predicting risk
- Authors Nayem and Biswas implement **Divide and Recombine** approach to fix speed & memory issues without loss of the accuracy of estimation.

Data description

Behavioral Risk Factor Surveillance System (BRFSS) data

- Datasets collected annually **from 2014 to 2024**
- A total of **2.4 million observations** with over **300 variables**
- Variables can be further classified into demographic, geographic, health, and lifestyle characteristics (not an exhaustive list of classifications)
- The data was acquired through the CDC's BRFSS surveys, conducted annually at the state and territory levels.

Domain	Variable	BRFSS code
Behavioral	Physical exercise	EXERANY2
	Smoking status	_RFSMOK3
	Alcohol consumption	DRNKANY6
	General health status	GENHLTH
Clinical	Cardiovascular disease	_MICHD
	Asthma	ASTHMA3
	Depression	ADDEPEV3
	Chronic kidney disease	CHCKDNY2
	Arthritis	HAVARTH4
Demographic	Age category	_AGEG5YR
	Sex	SEXVAR
	Marital status	MARITAL
	Educational attainment	_EDUCAG
Socioeconomic	Body mass index	_BMI5CAT
	Income level	_INCOMG1
Outcome	Diabetes status	DIABETE4

The Model: Divide and Recombine

1. Partition dataset into k smaller subsets
2. Fit logistic regressions for each subset
3. Combine each partition's estimate to approximate full sample MLE

The Model: Divide and Recombine

The Logistic regression model used by this paper is as follows:

- Outcome: Diabetes status(Yes/No)
- Predictors:
 - Behavioral: Exercise, Smoking Status, Alcohol Consumption, General Health Status
 - Clinical: Cardiovascular Disease, Asthma, Depression, Chronic Kidney Disease, Arthritis
 - Demographic: Age Category, Sex, Marital Status, Educational Attainment
 - Socioeconomic: Body Mass Index, Income Level

Paper conclusions

- D&R reduces computation time by 50% and memory use by 77%+ compared to the standard ``glm`` function
- For validation, the standard ``glm`` function and the ``drglm`` function with 25, 50, and 100 subsets were benchmarked on the BRFSS dataset and through MC simulations
- Applying D&R methodology to large BRFSS public health dataset shows it works at scale
- Paper created `'drglm'` function that the wider public can use...
- **And improve!**

Question(s) of interest

- Can we reproduce the paper's results?
- Can we make the function even faster and memory efficient?

Improving D&R implementation

Old workflow:

REPEATEDLY call `glm()` for each K, which rebuilds model matrices and increase memory use and therefore running time.

New workflow:

Build model matrices ONCE, then reuse it with `glm.fit()` to reduce memory use and running time accordingly.

Same concept, Different implementation

Reproducing the paper

1. Monte Carlo Simulations
2. Applying drglm to BRFSS data

Monte Carlo Simulations

Data Generating Process

- **Model:** logistic regression with 10 continuous predictors plus intercept
- **Sample size:** $n = 5,000,000$ observations per replication
- **Covariates:** $x_1, \dots, x_{10} \sim \text{Uniform}(0, 1)$, independently drawn
- **True coefficients:** $\beta = (2.00, 0.75, -1.25, 0.50, 0.60, 1.45, -0.40, 1.95, 0.55, 1.10, -0.80)^T$
(intercept first)
- **Outcome:** $y_i \sim \text{Bernoulli}(p_i)$, with 30–40% prevalence of $y = 1$
- **Monte Carlo replications:** $M = 500(\text{us})$ vs. $1000(\text{author})$
- **Seed:** 123

Monte Carlo Simulations

7 estimation methods were compared:

1. **CMLE (K = 1)**: MLE on full dataset using `glm()`
2. **D&R (K = 25)**: D&R across 25 subsets ($n_{\square} = 200,000$ per subset)
3. **D&R (K = 50)**: D&R across 50 subsets ($n_{\square} = 100,000$ per subset)
4. **D&R (K = 100)**: D&R for 100 subsets ($n_{\square} = 50,000$ per subset)
5. **D&R_New (K = 25)**: D&R across 25 subsets ($n_{\square} = 200,000$ per subset)
6. **D&R_New (K = 50)**: D&R across 50 subsets ($n_{\square} = 100,000$ per subset)
7. **D&R_New (K = 100)**: D&R for 100 subsets ($n_{\square} = 50,000$ per subset)

For each replication, we recorded parameter estimates $\beta^{(m)}$, standard errors $SE(\beta^{(m)})$, and computation time in seconds.

Results of MC simulations

Parameter	True Value	Mean Estimated Value			
		CMLE	D&R		
		$K = 1$	$K = 25$	$K = 50$	$K = 100$
Mean Absolute Bias	-	0.0002	0.0002	0.0002	0.0002

Author's mean absolute bias (1000 repetitions)

Parameter	True Value	D&R				D&R_new		
		Mean Estimated Value						
		$K = 1$	$K = 25$	$K = 50$	$K = 100$	$K = 25$	$K = 50$	$K = 100$
Mean Absolute Bias	-	0.0004	0.0005	0.0007	0.0011	0.0005	0.0007	0.0011

Our mean absolute bias (500 repetitions)

MC Simulation Memory Usage Comparison

Table 5: Peak memory usage comparison (in GB)

Method	$K = 1$	$K = 25$	$K = 50$	$K = 100$
Peak Memory (GB)	18.7	4.2	2.8	2.1
Reduction vs. CMLE (%)	–	77.5	85.0	88.8

Author’s memory usage (1000 repetitions)

Method	$K = 1$	$K = 25$	$K = 50$	$K = 100$
Peak Memory (GB)	4.22	2.88	2.83	2.81
Peak Memory _{new} (GB)	-	1.99	1.99	1.99
Reduction vs. CMLE (%)	-	31.81	32.85	33.41
Reduction_{new} vs. CMLE (%)	-	52.78	52.78	52.82

Our memory usage (500 repetitions)

MC Simulation Time Comparison

Method	$K = 1$	$K = 25$	$K = 50$	$K = 100$
CMLE	25.11	–	–	–
D&R	–	11.18	11.35	11.92
Time Reduction (%)	–	55.5	54.8	52.5

Author's times (1000 repetitions)

Method	$K = 1$	$K = 25$	$K = 50$	$K = 100$
CMLE	17.47			
D&R	17.16	19.25	18.91	
D&R_new	16.08	15.86	16.14	
Time Reduction (%)	1.74	-10.19	-8.27	
Time Reduction_new (%)	7.95	9.21	7.6	

Our times (500 repetitions)

Applying GLM to BRFSS Data

Data Preprocessing:

- Preprocessing code not provided by author, only analysis code
- Paper's final observation count ($n = 2,498,880$) was difficult to reproduce
 - Prior BRFSS years had different variable names for same question
 - Some years did not even ask the question!

Error in paper!

Label: Reported age in five-year age categories calculated variable

Section Name: Calculated Variables

Module Number: 7

Question Number: 11

Column: 1977-1978

Type of Variable: Num

SAS Variable Name: _AGE5YR

Question Prologue:

Question: Fourteen-level age category

Value	Value Label	Frequency	Percentage	Weighted Percentage
1	Age 18 to 24 Notes: 18 <= AGE <= 24	29,692	6.49	12.18
2	Age 25 to 29 Notes: 25 <= AGE <= 29	23,705	5.18	7.59
3	Age 30 to 34 Notes: 30 <= AGE <= 34	26,237	5.73	9.08
4	Age 35 to 39 Notes: 35 <= AGE <= 39	28,804	6.29	7.75
5	Age 40 to 44 Notes: 40 <= AGE <= 44	30,899	6.75	8.62
6	Age 45 to 49 Notes: 45 <= AGE <= 49	28,968	6.33	6.73
7	Age 50 to 54 Notes: 50 <= AGE <= 54	31,698	6.93	7.59
8	Age 55 to 59 Notes: 55 <= AGE <= 59	34,936	7.63	6.93
9	Age 60 to 64 Notes: 60 <= AGE <= 64	43,387	9.48	8.41
10	Age 65 to 69 Notes: 65 <= AGE <= 69	47,701	10.42	7.05
11	Age 70 to 74 Notes: 70 <= AGE <= 74	44,774	9.78	6.28
12	Age 75 to 79 Notes: 75 <= AGE <= 79	36,803	8.04	4.75
13	Age 80 or older Notes: 80 <= AGE <= 99	41,756	9.12	5.26
14	Don't know/Refused/Missing Notes: 7 <= AGE <= 9	8,310	1.82	1.79

Variable	Estimate			Std. error			z-value			95% CI (D&R)	
	D&R	glm	speedglm	D&R	glm	speedglm	D&R	glm	speedglm	Lower	Upper
(Intercept)	-4.991	-4.991	-4.991	0.037	0.037	0.037	-133.56	-133.56	-133.56	-5.064	-4.918
EXERANY22	0.187	0.187	0.187	0.005	0.005	0.005	40.71	40.71	40.71	0.178	0.196
_MICH2	-0.398	-0.398	-0.398	0.006	0.006	0.006	-70.62	-70.62	-70.62	-0.409	-0.387
ASTHMA32	-0.121	-0.121	-0.121	0.006	0.006	0.006	-21.43	-21.43	-21.43	-0.132	-0.110
ADDEPEV32	-0.072	-0.072	-0.072	0.005	0.005	0.005	-13.84	-13.84	-13.84	-0.082	-0.062
CHCKDNY22	-0.736	-0.736	-0.736	0.007	0.007	0.007	-102.42	-102.42	-102.42	-0.750	-0.722
MARITAL2	-0.101	-0.101	-0.101	0.006	0.006	0.006	-16.14	-16.14	-16.14	-0.113	-0.089
MARITAL3	0.025	0.025	0.025	0.006	0.006	0.006	3.90	3.90	3.90	0.012	0.037
MARITAL4	0.084	0.084	0.084	0.015	0.015	0.015	5.67	5.67	5.67	0.055	0.113
MARITAL5	0.040	0.040	0.040	0.007	0.007	0.007	5.45	5.45	5.45	0.026	0.054
MARITAL6	-0.091	-0.091	-0.091	0.013	0.013	0.013	-6.76	-6.76	-6.76	-0.117	-0.065
_EDUCAG2	-0.113	-0.113	-0.113	0.009	0.009	0.009	-12.86	-12.86	-12.86	-0.130	-0.096
_EDUCAG3	-0.058	-0.058	-0.058	0.009	0.009	0.009	-6.49	-6.49	-6.49	-0.075	-0.041
_EDUCAG4	-0.158	-0.158	-0.158	0.009	0.009	0.009	-17.19	-17.19	-17.19	-0.176	-0.140
DRNKANY62	0.466	0.466	0.466	0.004	0.004	0.004	106.88	106.88	106.88	0.457	0.474
_AGEG5YR2	0.288	0.288	0.288	0.030	0.030	0.030	9.69	9.69	9.69	0.230	0.346
_AGEG5YR3	0.688	0.688	0.688	0.027	0.027	0.027	25.52	25.52	25.52	0.635	0.741
_AGEG5YR4	1.143	1.143	1.143	0.025	0.025	0.025	45.00	45.00	45.00	1.093	1.193
_AGEG5YR5	1.578	1.578	1.578	0.025	0.025	0.025	64.14	64.14	64.14	1.530	1.626
_AGEG5YR6	1.986	1.986	1.986	0.024	0.024	0.024	81.99	81.99	81.99	1.939	2.033
_AGEG5YR8	2.470	2.470	2.470	0.024	0.024	0.024	103.91	103.91	103.91	2.423	2.517
_AGEG5YR10	2.704	2.704	2.704	0.024	0.024	0.024	114.23	114.23	114.23	2.658	2.750
_AGEG5YR11	2.812	2.812	2.812	0.024	0.024	0.024	118.33	118.33	118.33	2.765	2.859
_AGEG5YR12	2.835	2.835	2.835	0.024	0.024	0.024	118.05	118.05	118.05	2.788	2.882
_AGEG5YR13	2.571	2.571	2.571	0.024	0.024	0.024	106.16	106.16	106.16	2.524	2.618
_AGEG5YR14	2.264	2.264	2.264	0.033	0.033	0.033	68.48	68.48	68.48	2.199	2.329

Reproducing

Running `drglm()` vs. `glm()` on full dataset:

- `drglm()` was faster, but speedup was only 20% compared to the 50% cited in the paper
- Memory savings were not reproduced

Reproducing

Explaining speed discrepancy:

- Author's benchmark code not fully documented (for MC methods)
- `tic()` and `toc()` as benchmarking functions is flawed for benchmarking speed
 - Can be contaminated by background computer processes (not documented if controlled for)
 - Only one run is documented in the code (could just be luck!)
- Different computers with different specs/quality of PC parts can swing results
 - The same chip with same specifications can be of different quality
- We used `bench::mark()` function, which runs multiple iterations and can give summary stats but did not have the same speedup results

Reproducing

Explaining memory discrepancy:

- We were unable to reproduce memory findings
 - Used both their method (`pryr::mem_used`) and `bench::mark`
- Potentially due to background processes + garbage collection
 - GC is random in R

Order affects performance!!!

```
method                                     Function_Call
1 Full glm {fit_full_bench<-glm(fmla_bench,data=bench_dat,family=binomial())}
2 D&R K=100 reverse {local({b<-dr_oneshot_balanced(bench_dat,fmla_bench,k=100,gc_every=5)invisible(NULL)}}}
3 D&R K=50 reverse {local({b<-dr_oneshot_balanced(bench_dat,fmla_bench,k=50,gc_every=5)invisible(NULL)}}}
4 D&R K=25 reverse {local({b<-dr_oneshot_balanced(bench_dat,fmla_bench,k=25,gc_every=5)invisible(NULL)}}}
Elapsed_Time_sec Total_RAM_Used_MiB Peak_RAM_Used_MiB time_reduction_pct ram_reduction_pct speedup
1 58.50 1140.9 5334.3 0.00000 0.00000 1.000000
2 51.39 0.0 936.9 12.15385 82.43631 1.138354
3 52.30 0.0 1851.3 10.59829 65.29442 1.118547
4 51.16 0.0 3681.3 12.54701 30.98813 1.143471
> |
```

```
> bench_results
method                                     Function_Call Elapsed_Time_sec Total_RAM_Used_MiB
1 Full glm {fit_full_bench<-glm(fmla_bench,data=bench_dat,family=binomial())} 58.50 1140.9
2 D&R K=25 {b_dr25<-dr_oneshot(bench_dat,fmla_bench,k=25)} 48.86 0.0
3 D&R K=50 {b_dr50<-dr_oneshot(bench_dat,fmla_bench,k=50)} 54.23 0.0
4 D&R K=100 {b_dr100<-dr_oneshot(bench_dat,fmla_bench,k=100)} 65.18 0.0
Peak_RAM_Used_MiB time_reduction_pct ram_reduction_pct speedup
1 5334.3 0.000000 0.00000 1.000000
2 752.9 16.478632 85.88568 1.1972984
3 386.3 7.299145 92.75819 1.0787387
4 203.3 -11.418803 96.18882 0.8975146
> |
```

Conclusion

- We are able to meaningfully improve and speed up the drglm function
- We can reproduce *parts* of the paper, but the author's benchmarking is undocumented, misleading, or flawed in method
- The preprocessing of the paper is entirely omitted, which makes it difficult to fully reproduce
 - Future papers should take more care in benchmarking methodology and documenting data cleaning